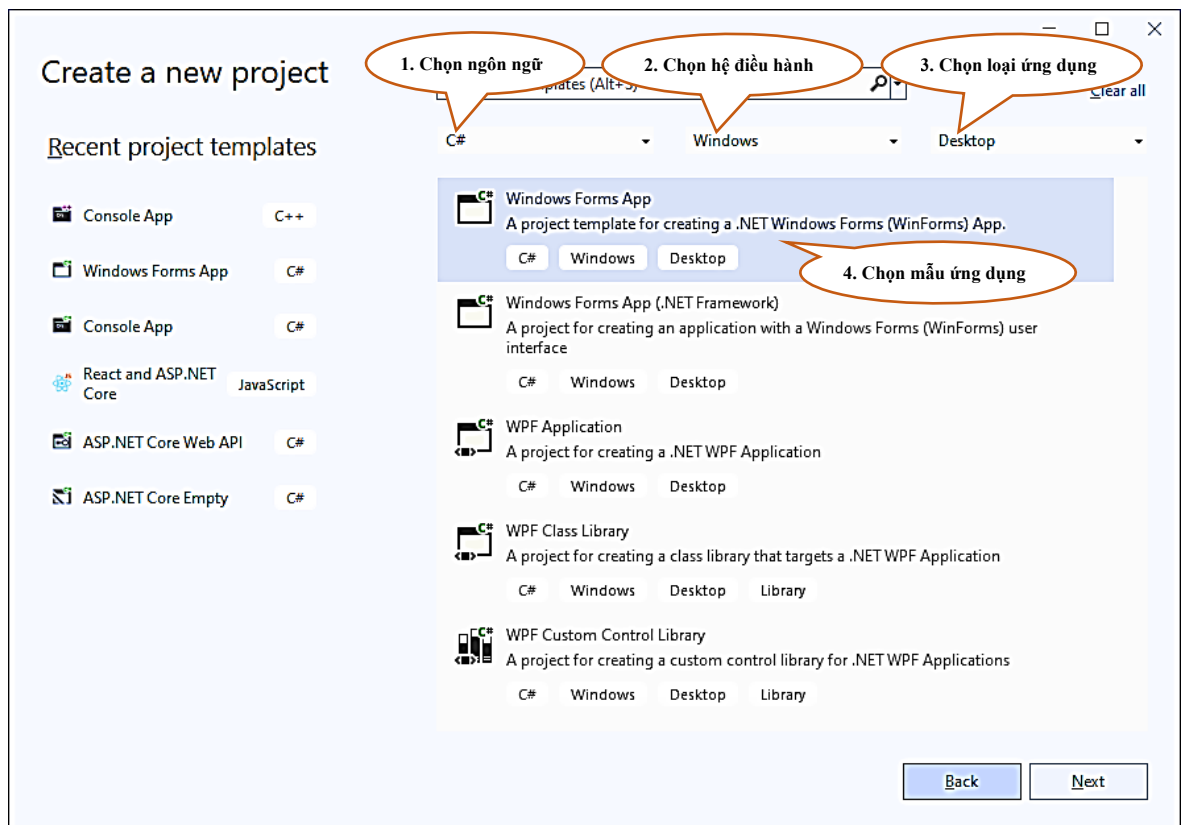


Chương 3. THIẾT KẾ GIAO DIỆN HỆ THỐNG PHẦN MỀM

1. Giao diện

1.1. Tạo ứng dụng có giao diện cửa sổ

Khởi chạy Visual studio, nhấp chọn Create a new project:



Chọn C#, Windows, Desktop, Windows Forms App. Nhấp Next.

Lưu ý: Nếu chọn mẫu Windows Forms App (.Net Framework), sau đó bạn có thể chọn phiên bản cũ (.Net Framework 2.0 đến 4.8).

LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh

Configure your new project

Windows Forms App C# Windows Desktop

Project name 1. Tên dự án
WinFormsApp5

Location 2. Nơi lưu
C:\Users\WIN10PC\source\repos

Solution name 3. Tên giải pháp, thường trùng tên dự án
WinFormsApp5

☐ Place solution and project in the same directory

Project will be created in "C:\Users\WIN10PC\source\repos\WinFormsApp5\WinFormsApp5\"

Back Next

Đặt tên dự án, giải pháp (ứng dụng), chọn nơi lưu. Next

Additional information

Windows Forms App C# Windows Desktop

Framework
.NET 8.0 (Standard Term Support)

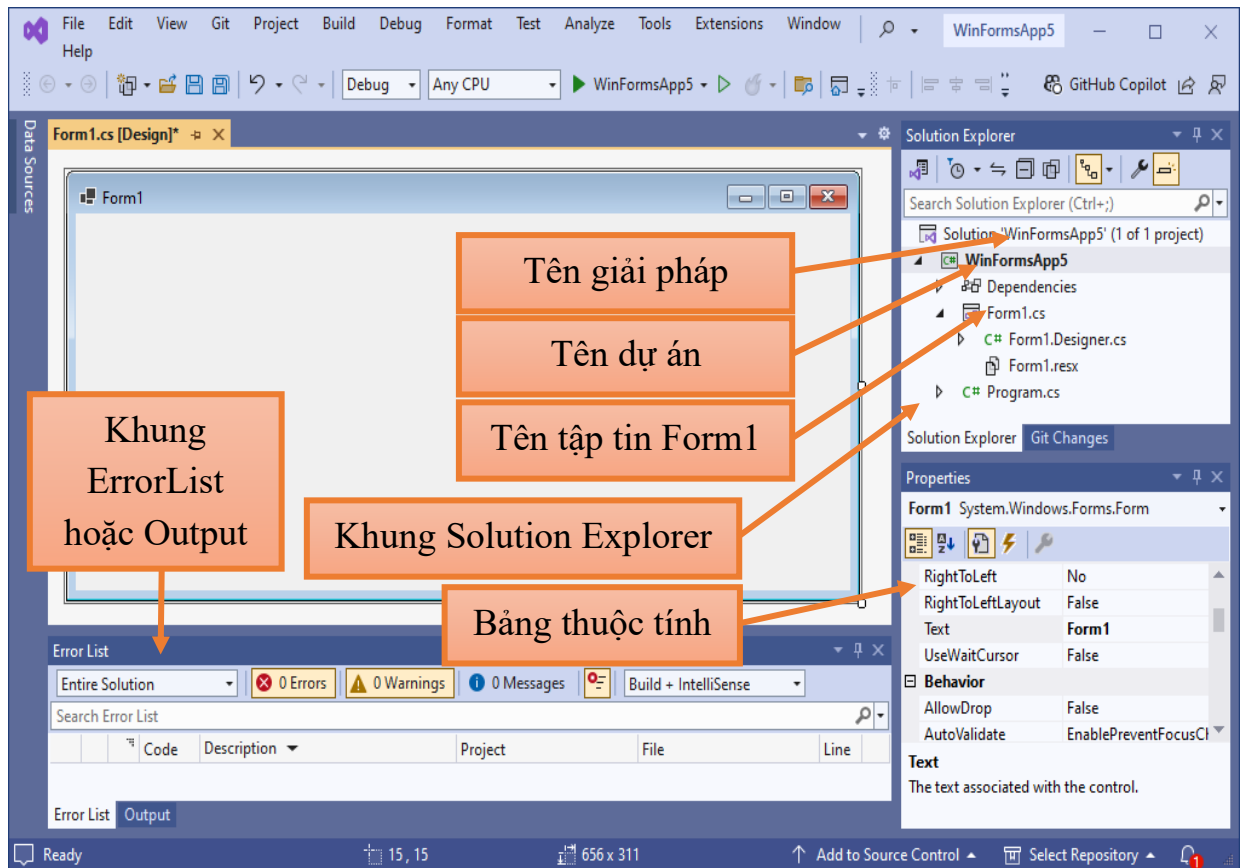
Back Create

Chọn phiên bản .NET 8 (hoặc cao hơn). Nhấp Create.

Bạn sẽ thấy trong cửa sổ xuất hiện một form trống (Form1) để thiết kế giao diện ứng dụng.

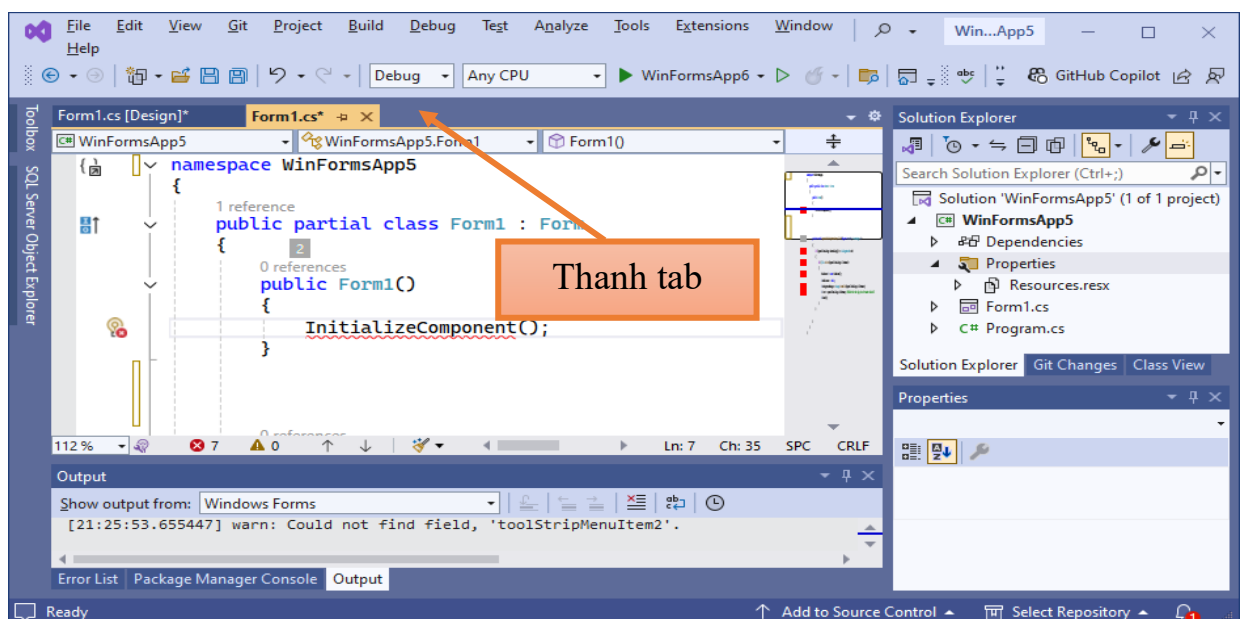
LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh



Nếu không thấy các khung, bảng, thì vào lệnh View nhập lệnh tương ứng.

Nhấp phải chuột vào tên tập tin Form1.cs, nhấp lệnh View Code để mở cửa sổ chương trình (code) của Form1, còn nhấp lệnh View Designer để mở cửa sổ thiết kế Form1. Trên thanh tab hiển thị tên các cửa sổ đang mở (Form1.cs [Design] là tên cửa sổ thiết kế, Form1.cs là tên cửa sổ code), có thể nhấp vào tên để chuyển sang cửa sổ khác đang mở.



1.2. Thuộc tính đối tượng

Trong một dự án Windows Form, để xây dựng ứng dụng bạn cần tạo lập và sử dụng các đối tượng.

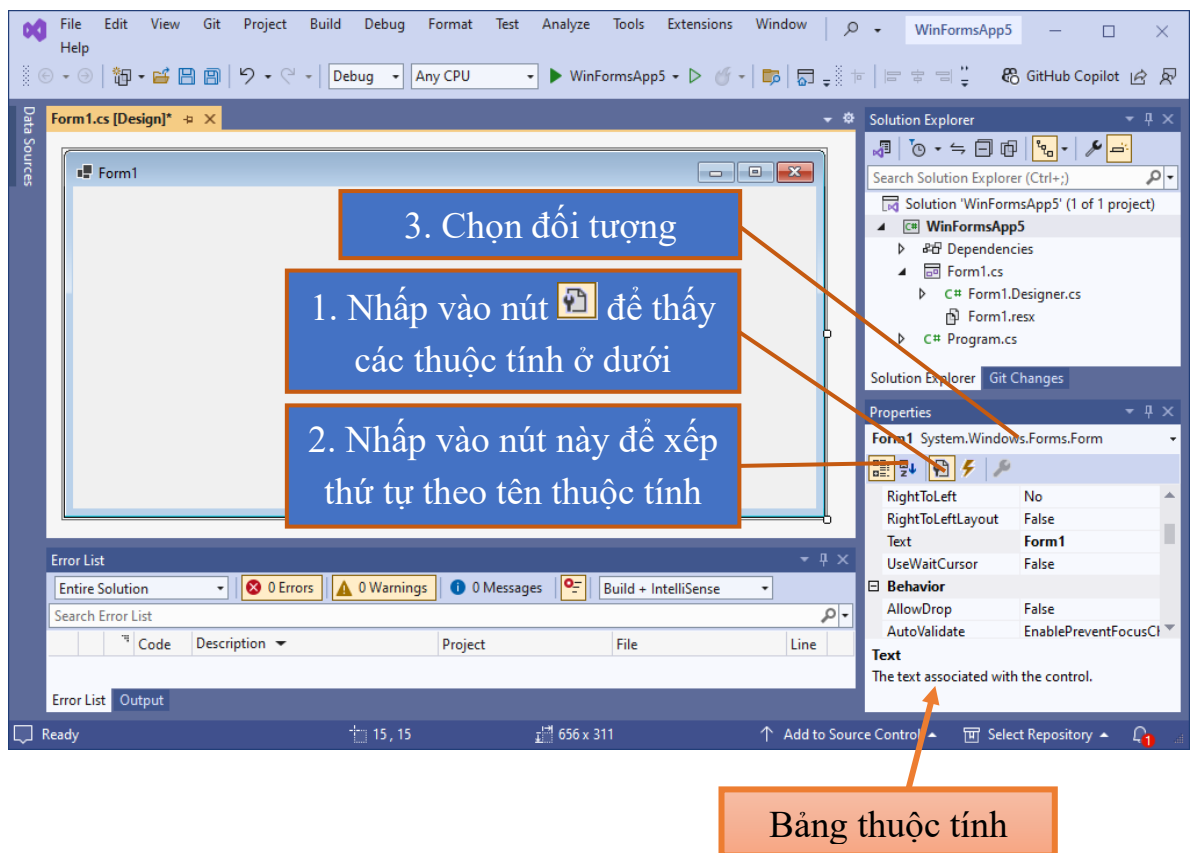
Ví dụ: Các đối tượng trong ứng dụng như Form, Label, Textbox,...

Mỗi đối tượng Form, Label, Textbox,... được gọi là điều khiển (control).

Mỗi form và đối tượng điều khiển khác có các thuộc tính (property) nào đó để cho biết và/hoặc thay đổi các đặc điểm của đối tượng.

Ví dụ: Form có thuộc tính BackColor cho biết màu nền hiện hành hoặc cho thay đổi màu nền của nó.

Sau đó chọn đối tượng (form /button /...) trên cửa sổ thiết kế hoặc ở bảng thuộc tính (nếu bảng này chưa mở thì nhấp phải chuột vào đối tượng, nhấp lệnh *Properties*), nhấp vào nút Properties  để thấy danh sách các tên các thuộc tính. Trong bảng này, ta có thể xem giá trị hiện hành của một thuộc tính nào đó và có thể thay đổi sang giá trị mới.



Tuy nhiên, bảng thuộc tính không hiển thị đầy đủ mọi thuộc tính của đối tượng. Có những thuộc tính ta phải truy xuất bằng chương trình.

1.3. Các thuộc tính thông dụng của Form

- Name: Tên của Form.
- Text: Tiêu đề form.
- WindowState: Kích thước form khi chạy ứng dụng.
 - 0. Normal - Theo chỉ định lúc thiết kế.
 - 1. Minimized - Cực tiểu.
 - 2. Maximized - Cực đại.
- FormBorderStyle: Loại khung của form
 - + None - không có khung và thanh tiêu đề.
 - + FixedSingle - khung đơn, không thể dùng chuột thay đổi kích thước khung.
 - + Fixed3D - khung đôi, không thể dùng chuột thay đổi kích thước khung.
 - + FixedDialog - form kiểu hộp thoại. Phải đóng form mới được chuyển sang form khác của ứng dụng.
 - + Sizable - Form bình thường.

1.4. Sự kiện

Sự kiện (event) là một tác động lên đối tượng.

Khi chạy một ứng dụng Windows, nó thường trưng ra một cửa sổ (form). Khi bạn nhấp chuột vào cửa sổ, thì bạn đã tạo ra một sự kiện là Click (nhấp chuột) của form. Nếu bạn nhấp chuột vào một đối tượng khác trên form, chẳng hạn nhấp vào nút lệnh trên form, thì bạn cũng tạo ra một sự kiện là Click, nhưng sự kiện này là của nút lệnh.

✿ Các sự kiện của form:

- ✓ Load là sự kiện xảy ra khi nạp form này vào bộ nhớ, cụ thể là
 - Khi chạy ứng dụng và hiển thị form này
 - hoặc khi hiển thị form lần đầu bằng: phương thức Show() của form hay đặt thuộc tính Visible = true cho form. Hiển thị lần đầu có nghĩa là: trước đó form chưa được nạp vào bộ nhớ hoặc đã nạp và sau đó bị đóng, rồi được bạn hiển thị nó lên.

LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh

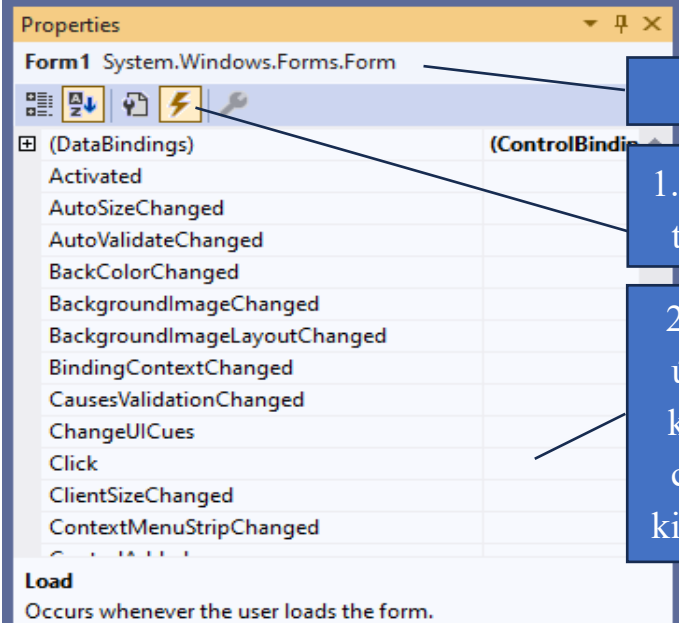
Trong lúc nạp, form chưa hiển thị hẳn lên màn hình.

Thường sử dụng sự kiện Load để gán các giá trị cho biến dùng chung trong form hoặc khởi tạo các đối tượng của form.

- ✓ DoubleClick là sự kiện xảy ra khi nhấp đúp chuột vào form.
- ✓ FormClosing là sự kiện xảy ra trước khi gỡ form khỏi bộ nhớ, cụ thể là
 - Khi đóng form: nhấp ☐ hay ấn Alt+F4
 - hoặc đóng form bằng phương thức Close() của form.
- ✓ FormClosed là sự kiện xảy ra khi gỡ form khỏi bộ nhớ (khi đóng form), sau sự kiện FormClosing.

Bình thường khi sự kiện xảy ra thì ứng dụng thực hiện công việc mặc định nào đó của sự kiện hoặc không làm gì cả. Nếu bạn muốn khi sự kiện xảy ra và ứng dụng phải làm gì đó theo ý bạn, thì bạn phải viết chương trình cho nó.

Để viết chương trình cho mỗi sự kiện của một đối tượng, bạn cần lặp lại trình tự sau: Mở cửa sổ thiết kế form. Sau đó chọn đối tượng (form /button /...) trên cửa sổ thiết kế hoặc ở bảng thuộc tính (*nếu bảng này chưa mở thì nhấp phải chuột vào đối tượng, nhấp lệnh Properties*), nhấp vào nút Events  để thấy danh sách các tên sự kiện. Muốn viết chương trình cho sự kiện nào, chẳng hạn sự kiện Click, thì nhấp đúp vào sự kiện đó như hình sau, để vào cửa sổ chương trình của form. Sau đó bạn viết chương trình vào hàm sự kiện tương ứng.



3. Chọn đối tượng

1. Nhấp vào nút  để thấy tên các sự kiện

2. Nhấp đúp vào đây ứng với Click (hoặc khác) nếu muốn viết chương trình cho sự kiện Click (hoặc khác)

Load
Occurs whenever the user loads the form.

LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh

Ví dụ: Chương trình cho các hàm sự kiện Click, DoubleClick, Load, FormClosing, FormClosed của form (*Giả sử tên (name) của form là Form1*).

```
private void Form1_Click(object sender, EventArgs e)
{
    BackColor = Color.Red;
}
private void Form1_DoubleClick(object sender, EventArgs e)
{
    MessageBox.Show("Đừng nhấp nữa", "Chào", MessageBoxButtons.OK);
}
private void Form1_Load(object sender, EventArgs e)
{
    MessageBox.Show("Chào các bạn", "Chào", MessageBoxButtons.OK);
}
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    if (MessageBox.Show("Tôi về nhé", "Chào", MessageBoxButtons.YesNo)
        == DialogResult.No)
    {
        e.Cancel = true;
    }
}
private void Form1_FormClosed(object sender, FormClosedEventArgs e)
{
    MessageBox.Show("Tôi về rồi", "Chào", MessageBoxButtons.OK);
}
```

Sự kiện Click sẽ đổi màu nền của form thành màu đỏ bằng câu lệnh:

BackColor = Color.Red

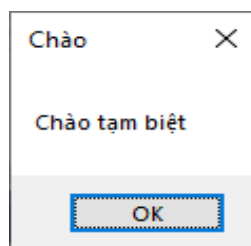
⇒ Ta có thể dùng câu lệnh để lấy hoặc thay đổi giá trị các thuộc tính của một đối tượng.

MessageBox.Show là lệnh hiển thị hộp thoại.

- Cú pháp: MessageBox.Show ("thông báo" , "tiêu đề", nút)

- MessageBox.Show ("Chào tạm biệt", "Chào", MessageBoxButtons.OK)

sẽ hiển thị hộp thoại:



Ở sự kiện FormClosing, ta có thể hủy sự kiện đóng form bằng câu lệnh:

e.Cancel = true

Ví dụ: Khi đóng form sẽ hiển thị hộp thoại hỏi "Bạn thật sự muốn đóng không?". Nếu nhấp Yes thì đóng. Nếu nhấp No thì không đóng. Chương trình cho Form_FormClosing

```
if (MessageBox.Show("Bạn thật sự muốn đóng không?", "Chào", MessageBoxButtons.YesNo)
    == DialogResult.No)
    e.Cancel = true;
```

Quan trọng: **e.Cancel = true không kết thúc sự kiện**, nó vẫn chạy các câu lệnh tiếp theo nếu có của sự kiện. Khi đến cuối phương thức, nó rời khỏi phương thức với sự kiện “đóng form” không hoàn thành, nên vẫn ở lại form.

1.5. Các điều khiển thường dùng

a) Cách tạo đối tượng trên form

Để tạo một đối tượng điều khiển trên form, ta nhấp đúp một công cụ control tương ứng trên hộp công cụ. Đối tượng mới tạo sẽ xuất hiện trên form. Ta có thể dùng chuột hoặc cửa sổ thuộc tính để thay đổi vị trí và kích thước của đối tượng.

b) Các thuộc tính của các đối tượng

- Name: Tên của đối tượng.
- Enabled: True – Cho phép đối tượng hoạt động.
- Font: định dạng chữ cho đối tượng (font, cỡ, đậm, nghiêng, gạch đứt,...)
- ForeColor: Màu chữ của đối tượng.
- Text: Chữ trên đối tượng.
- TabIndex: Thứ tự nhảy vào đối tượng khi ấn phím Tab.
- TabStop: True – Cho phép con trỏ nhảy vào đối tượng khi ấn phím Tab.
- Visible: True – Hiển thị đối tượng.
- BackgroundImage: Hình nền của đối tượng.
- BackgroundImageLayout: chọn kiểu trình bày hình nền.
 - + None – Loại bỏ hình nền.
 - + Stretch – Co giãn hình nền bằng với đối tượng.

c) Label (nhãn) là đối tượng hiển thị chuỗi ký tự.

d) Button là nút lệnh.

e) Textbox là ô nhập dữ liệu.

- MaxLength: Số lượng ký tự tối đa có thể nhập vào ô.
- TextAlign: canh chữ qua trái / giữa / phải
- ReadOnly: True - Nội dung ô chỉ đọc (không cho phép nhập vào ô).
- RightToLeft: Yes - Ký tự nhập sẽ xuất hiện từ cạnh phải.
- UseSystemPasswordChar: True - sử dụng ký tự • làm ký tự hiển thị.

f) PictureBox là ô chứa hình.

- Image: Hình mà ô sẽ chứa (khác với hình nền của ô)
- SizeMode:
 - + Normal – Bình thường.
 - + StretchImage – co giãn hình bằng với ô chứa.
 - + Zoom – Thay đổi kích thước hình theo tỉ lệ bằng nhau ở 2 chiều ngang và dọc để hình có kích thước lớn nhất.
- + CenterImage – Đặt hình vào giữa ô chứa.
- + AutoSize – Thay đổi kích thước ô chứa vừa với hình. Do đó không thay đổi kích thước ô được.

● Tải hình vào ô PictureBox (giả sử name=pictureBox1):

```
if (File.Exists("tên_tập_tin"))
```

```
    pictureBox1.ImageLocation = tên_tập_tin;
```

Tên_tập_tin có dạng "c:\images\hoa.bmp" hoặc @"c:\images\hoa.bmp" hoặc "c:/images/hoa.bmp". Các kiểu tập tin: jpeg, jpg, jpe, png, gif, ico, bmp, tif, tiff, jfif, dib, ...

● Xóa hình khỏi ô: pictureBox1.ImageLocation = "";

1.6. Phương thức (method)

Phương thức (Hành vi) là hành động mà đối tượng có thể thực hiện.

Các đối tượng thường có các phương thức sau:

- + Focus: Đặt con trỏ đến đối tượng.
- + Hide: Ẩn đối tượng (không gỡ khỏi bộ nhớ).

LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh

+ Show: Hiện đối tượng.

Ta sử dụng các phương thức trong chương trình.

VD: Câu lệnh để ẩn nút lệnh (name = button1): `button1.Hide()`

Câu lệnh để hiện nút lệnh: `button1.Show()`

VD: Câu lệnh để đặt con trỏ vào Textbox (name = textBox1):

`textBox1.Focus()`

Kết thúc ứng dụng là: `Application.Exit()`.

Đóng form: Sử dụng phương thức `Close()` hoặc nhấp nút đóng . Xảy ra sự kiện `FormClosing`, rồi `FormClosed`. Form sẽ được gỡ ra khỏi bộ nhớ.

Hiện thị form: Sử dụng phương thức `Show()` hoặc đặt thuộc tính `Visible = true`. Form được nạp vào bộ nhớ và xảy ra sự kiện `Load` nếu đây là lần đầu hiển thị form.

Ẩn form (chưa đóng): Sử dụng phương thức `Hide()` hoặc đặt thuộc tính `Visible = false`. Vẫn có thể tham chiếu biến, control của nó.

1.7. Ứng dụng minh họa

Tạo ứng dụng Windows Form tên là `TinhToan`.

- Thuộc tính của Form:
 - Name = `fMain`
 - `BorderStyle = FixedSingle` (không thể dùng chuột thay đổi kích thước form)
 - Text = Calculator

- Name của các ô Textbox:

- `txtBk`

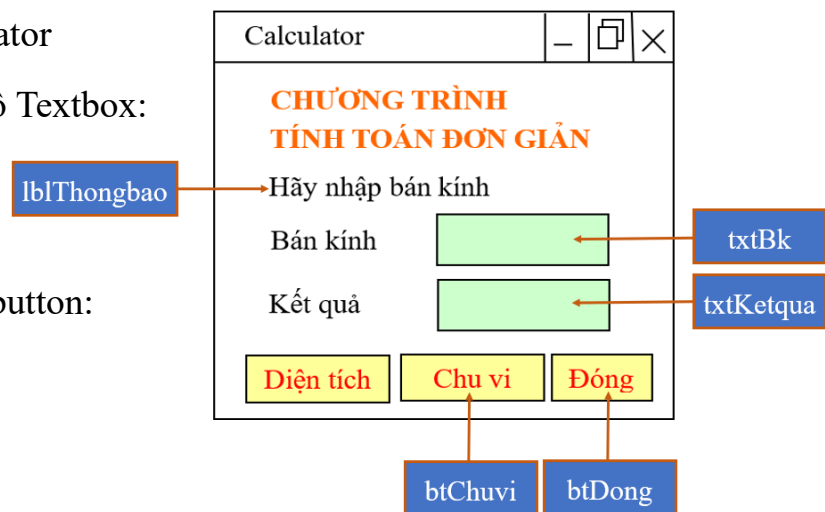
- `txtKetqua`

- Name của các button:

- `btDientich`

- `btChuvi`

- `btDong`



- Label với Name là `lblThongbao` và Text là “Hãy nhập bán kính”.
- Enabled của `txtKetqua` là False

LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh

- TabIndex của txtBk, btDientich, btChuvi lần lượt là 0, 1, 2
- Chương trình của form:

namespace Tinhtoan

```
{  
    public partial class fMain : Form  
    {  
        const decimal Pi = 3.1415M;  
        public fMain()  
        {  
            InitializeComponent();  
        }  
        private void btChuvi_Click(object sender, EventArgs e)  
        {  
            txtKetqua.Text = (2*Convert.ToDecimal(txtBk.Text)*Pi).ToString();  
            txtBk.Focus();  
        }  
        private void btDientich_Click(object sender, EventArgs e)  
        {  
            txtKetqua.Text = (Convert.ToDecimal(txtBk.Text)  
                *Convert.ToDecimal(txttBk.Text) * Pi).ToString();  
            txtBk.Focus();  
        }  
        private void void btDong_Click(object sender, EventArgs e)  
        {  
            Close();  
        }  
    }  
}
```

1.8. Kiểm tra dữ liệu

Tạo thêm lớp tên Utility để chứa các phương thức, thuộc tính, trường hữu ích sẽ được sử dụng trong ứng dụng.

LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh

Nhấp phải chuột vào tên dự án, nhấp lệnh Add \ Class. Đặt tên tập tin là Utility.cs. Soạn chương trình sau cho lớp Utility:

```
using System.Windows.Forms;

namespace Tinhtoan
{
    class Utility
    {
        public static string CheckDecimal(Control obj, string s)
        {
            decimal t;
            if (obj.Text == "")
                return "Bạn chưa nhập vào ô " + s;
            else
                if (!decimal.TryParse(obj.Text, out t))
                    return "Dữ liệu " + obj.Text + " trong ô " + s + " chưa là số?";
                else
                    return "decimal";
        }
    }
}
```

Phương thức CheckDecimal nhận một điều khiển obj (Textbox,...), rồi kiểm tra dữ liệu trong điều khiển obj có phải là số kiểu decimal hay không. Phương thức trả về chuỗi "decimal" nếu dữ liệu là kiểu decimal. Từ khóa **static** cho phép sử dụng phương thức qua tên lớp.

Chương trình của sự kiện click của nút tính Chu vi:

```
txtKetqua.Text = "";

lblThongbao.Text = Utility.CheckDecimal(txtBk, "bán kính");

if (lblThongbao.Text == "decimal")
    if (Convert.ToDecimal(txtBk.Text) >= 0)
    {
        txtKetqua.Text = (2 * Convert.ToDecimal(txtBk.Text) * Pi).ToString();
    }
}
```

LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh

```
lblThongbao.Text = "Hãy nhập bán kính";  
}  
else  
    lblThongbao.Text = "Sai! Hãy nhập số >=0 cho bán kính?";  
txtBk.Focus();
```

1.9. Các điều khiển khác

a. RichTextBox

Đối tượng này cũng giống như TextBox, nhưng có nhiều tính năng hơn: chứa đến 2147483647 ký tự, định dạng chữ cho từng phần văn bản, chứa hình trong văn bản,...

b. Tooltip

Cho phép cài đặt **chuỗi thông báo** hoặc **chuỗi gợi ý**.

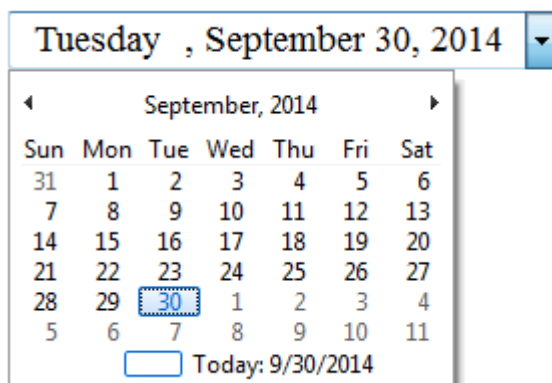
Chuỗi thông báo xuất hiện khi thực hiện phương thức Show của đối tượng Tooltip. Phương thức gồm chuỗi cần hiển thị, đối tượng làm mốc để hiển thị chuỗi, vị trí xuất hiện chuỗi (tương đối so với mốc), khoảng thời gian hiển thị.

VD: `toolTip1.Show("Bán kính>=0?", txtBk, 0, -20, 1000);`

Lưu ý: Khi ta nhấp đúp công cụ Tooltip trên hộp công cụ để đưa vào form. Đối tượng Tooltip không nằm trên bề mặt form mà nằm ở vùng bên dưới form.

c. DateTimePicker

Đây là điều khiển cho phép chọn ngày tháng năm thay vì nhập.



Định dạng ngày/tháng/năm:

- Format : Custom
- CustomFormat: dd/MM/yyyy

LẬP TRÌNH WINDOWS

GV: Hồ Ngọc Thanh

Lưu ý: MM và mm sẽ khác nhau.

Thuộc tính Value cho biết ngày tháng năm được chọn. Value vẫn ra giá trị kiểu long date dù đã chọn Custom và CustomFormat. Có thể đưa giá trị dạng long date vào cột kiểu datetime của bảng dữ liệu.

Lấy giá trị: `DateTime date = dateTimePicker1.Value;`

Đổi sang chuỗi: `date.ToShortDateString();`

Hoặc `dateTimePicker1.Value.Date.ToString("MM-dd-yyyy")`

Lấy năm, tháng, ngày riêng lẻ:

`dateTimePicker1.Value.Year`

`dateTimePicker1.Value.Month`

`dateTimePicker1.Value.Day`

Gán giá trị cho `dateTimePicker1`:

`dateTimePicker1.Value = new DateTime(yyyy,mm,dd,hh,mi,0);`

d. Masked Textbox

Đây là điều khiển cho phép nhập dữ liệu theo khuôn nhập của nó.

Thuộc tính Mask xác định khuôn nhập dữ liệu.

VD: Mask để nhập tên (tối thiểu 1 ký tự, tối đa 10 ký tự, L → ký tự chữ bắt buộc, ? → ký tự chữ tùy chọn, > là in hoa, < là in thương) là `>L<?????????`

VD: Mask để nhập ngày tháng năm (0 → ký tự số bắt buộc) `00/00/0000`

e. Checkbox

CheckBox là đối tượng ô kiểm. Nó có 3 trạng thái: mở ☒, tắt ☐ và xám ☐

Thuộc tính Checked chỉ truy xuất được 2 giá trị sau:

- true ⇔ Trạng thái mở ☒.

- false ⇔ Trạng thái tắt ☐.

Thuộc tính CheckState truy xuất được 3 giá trị sau:

- CheckState.Checked ⇔ Trạng thái mở.

- CheckState.Unchecked ⇔ Trạng thái tắt.

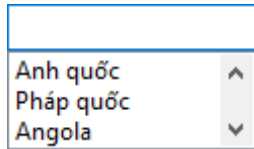
- CheckState.Indeterminate ⇔ Trạng thái xám ☐

f. ComboBox

ComboBox là hộp danh sách, liệt kê các giá trị (các mục) để người dùng chọn mục. Nó có ô nhập để người dùng nhập thay vì chọn.

Có 3 loại ComboBox:

- Simple: hiển thị luôn danh sách. Nó có ô nhập



- Dropdown: phải nhấp vào nút mới xổ ra danh sách. Nó có ô nhập



- DropDownList: Hình dạng như loại 2, nhưng không cho sử dụng ô nhập.

Các thuộc tính:

- DropDownStyle: loại Combobox.
- MaxLength: Số lượng ký tự có thể nhập vào ô nhập.
- SelectedIndex: chỉ mục của mục được chọn. Chỉ mục đầu tiên là 0. Nếu không có mục được chọn thì chỉ mục là -1.
- Text: Tên mục của mục được chọn, có phân biệt chữ hoa, chữ thường. Nếu nhập vào ô nhập thì Text là chuỗi đã nhập. Nếu không có mục được chọn hoặc ô nhập rỗng thì Text là null.

Tạo danh sách mục bằng thiết kế: Trong cửa sổ thuộc tính của ComboBox, ở thuộc tính Item nhấp vào nút , rồi nhập các mục cho ComboBox.

Tạo danh sách mục bằng đoạn lệnh:

```
comboBox1.Items.Add("Anh");  
comboBox1.Items.Add("Pháp");  
comboBox1.Items.Add("Đức");
```

Xóa mục đang chọn: `comboBox1.Items.Remove(comboBox1.SelectedItem)`

Sự kiện Click xảy ra khi nhấp vào ComboBox.

Sự kiện `SelectIndexChanged` xảy ra khi nhấp chọn một mục hoặc dùng phím mũi tên để chọn mục.

BÀI TẬP

1. Xây dựng ứng dụng của ví dụ: Tính toán chu vi, diện tích.

Nộp LMS: tập tin word stt.docx chứa

- Hình chụp cửa sổ đang chạy ứng dụng có kết quả tính diện tích.

- Utility.cs:

`//code`

- fMain.cs:

`//code của form`